# MISA: Minimal Instruction Set Architecture

Jonathan Uhler

2026-09-16

# Table of contents

**Preface** **5**

**1 Instruction Set Architecture** **6**
1.1 Conformance Language . . . 6
1.2 General Purpose Registers . . . 7
1.3 Control and Status Registers . . . 8
1.3.1 `SADDR` . . . 8
1.3.2 `RADDR` . . . 8
1.3.3 `FLAGS` . . . 9
1.3.4 `CAUSE` . . . 10
1.3.5 `EXTNS` . . . 12
1.4 Data Formats and Alignments . . . 12
1.5 Memory . . . 13
1.5.1 Required Memory Regions . . . 13
1.5.2 Recommended Memory Map . . . 13
1.5.3 Stack Conventions . . . 14
1.6 Reset . . . 14
1.7 Faults . . . 15
1.8 Exceptions and Interrupts . . . 15
1.9 Base Instructions . . . 15
1.9.1 `HALT` . . . 16
1.9.2 `ADD` . . . 16
1.9.3 `ADC` . . . 17
1.9.4 `SUB` . . . 17
1.9.5 `SBB` . . . 18
1.9.6 `AND` . . . 18
1.9.7 `OR` . . . 19
1.9.8 `XOR` . . . 19
1.9.9 `RRC` . . . 20
1.9.10 `SET` . . . 20
1.9.11 `LD` . . . 21
1.9.12 `ST` . . . 21
1.9.13 `RSR` . . . 22
1.9.14 `WSR` . . . 22
1.9.15 `JAL` . . . 23

1.9.16 JMP . . . 23
1.10 Pseudo Instructions . . . 25
1.10.1 ADD2 . . . 25
1.10.2 AND2 . . . 25
1.10.3 CALL . . . 26
1.10.4 CLR . . . 26
1.10.5 CMP . . . 26
1.10.6 GOTO . . . 26
1.10.7 JALI . . . 27
1.10.8 JMPI . . . 27
1.10.9 MOV . . . 27
1.10.10 MOV2 . . . 28
1.10.11 NOP . . . 28
1.10.12 OR2 . . . 28
1.10.13 POP . . . 28
1.10.14 POP2 . . . 29
1.10.15 PUSH . . . 29
1.10.16 PUSH2 . . . 29
1.10.17 RET . . . 29
1.10.18 RRC2 . . . 30
1.10.19 SET2 . . . 30
1.10.20 SUB2 . . . 30
1.10.21 XOR2 . . . 31
1.11 Architecture Extensions . . . 31
1.11.1 Dynamic Hardware Extension . . . 31
1.11.2 System Call Extension . . . 31
1.11.3 Interrupt Extension . . . 33
1.11.4 Privilege Extension . . . 38

**2 Toolchain** **40**
2.1 Assembler . . . 40
2.1.1 Assembler Pipeline . . . 40
2.1.2 Assembler Command Line . . . 41
2.1.3 Assembler Semantics . . . 41
2.1.4 MISA Linkable Format . . . 43
2.2 Linker . . . 46
2.2.1 Memory Map Files . . . 46
2.3 Other Tools . . . 46
2.3.1 Preprocessor . . . 47
2.3.2 Archiver . . . 48
2.3.3 Name Lister . . . 48
2.3.4 Object Dumper . . . 49

**3 Simulator And Hardware Reference** **51**
3.1 Functional Simulator . . . . . 51
3.1.1 Loading And Reset . . . . . 51
3.1.2 Instruction Execution . . . . . 51
3.1.3 Address Space . . . . . 52
3.2 Interactive Debugger . . . . . 52
3.3 Hardware Reference . . . . . 53

# Preface

Minimal Instruction Set Architecture (MISA) is a limited instruction set and computer architecture designed for educational purposes. The ultimate design goal is to apply a large set of the knowledge from the average computer science undergraduate degree into one project that includes:

- Computer architecture
- Computer system design
- Logic design and computer engineering
- Data structures and algorithms
- Programming language theory
- Computability theory
- Maybe some basics on operating systems
- And various mathematical concepts

The core component of MISA is the namesake minimal instruction set architecture itself. While unlikely, a goal for the project is to tape out a very simple (i.e. single-cycle) processor that uses the MISA architecture. Given that, the ISA is intended to be as limited as possible, hopefully in a way that optimizes for the chip area constraints enforced by hobbist tapeout programs. The ISA from a programmer's view is fully defined in Chapter 1 of this document.

The MISA project also includes a toolchain for creating programs to run on the MISA architecture. This includes an assembler, linker, and some object dumping/analysis tools. Some notes on the software design of these tools, in addition to their usage, are available in Chapter 2.

A software simulator is also available as the functional reference for the architecture. The intent of the simulator is to provide a full reference implementation for the functionality of the MISA instructions, so it defines how a conforming processor executes a program. Notes on the simulator and its interactive debugger are in Chapter 3. That chapter also holds a placeholder for the planned hardware reference, which will document the intended silicon design.

# 1 Instruction Set Architecture

This chapter describes the MISA instruction set architecture, including the base instruction set, optional assembler extensions, special registers and hardware data structures, and other implementation-agnostic features.

The MISA architecture is an 8-bit architecture with up to a 16-bit address bus to interface with externally connected memory. Instructions are fixed-width 16-bit doublewords in little-endian byte order.

The information presented in this chapter is referred to as the MISA core architecture. The last section of this chapter enumerates several optional extensions to the core architecture, although an implementation is considered conforming if it implements at least the core architecture.

## 1.1 Conformance Language

For an implementation of the MISA architecture to be considered "conforming" to this chapter, there are several requirements it needs to respect, although some liberty is given to the implementation.

The following words used in this chapter have specific meanings when describing whether and how a conforming implementation implements a part of the architecture:

- "MUST": A conforming implementation is required to include or omit the specified feature/behavior. If the implementation does not respect the requirement, it is not considered conforming to this specification.
- "SHOULD": A conforming implementation can choose whether or not to include the specified feature, but this specification recommends its inclusion. Implementations that do not include the feature are still considered conforming, but some aspect of the implementation may suffer.
- "MAY": A conforming implementation can choose whether or not to include the specified feature. Both an implementation that does, and an implementation that does not include the feature will be considered conforming. This specification makes no recommendation on the inclusion of the feature.

This normative terminology will appear in ALL CAPS throughout this chapter.

## 1.2 General Purpose Registers

The general-purpose register file MUST contain sixteen, 8-bit registers for general computation and data storage, defined by the following ABI:

| ABI Name | ABI Purpose | Wide ABI Name | Encoding |
|---|---|---|---|
| `R0` | Hard-wired zero | N/A | 0000 |
| `RA` | Argument/temporary | `RAB` | 0001 |
| `RB` | Argument/temporary | | 0010 |
| `RC` | Argument/temporary | `RCD` | 0011 |
| `RD` | Argument/temporary | | 0100 |
| `RE` | Argument/temporary | `REF` | 0101 |
| `RF` | Argument/temporary | | 0110 |
| `RU` | Saved | `RUV` | 0111 |
| `RV` | Saved | | 1000 |
| `RW` | Saved | `RWX` | 1001 |
| `RX` | Saved | | 1010 |
| `RY` | Saved | `RYZ` | 1011 |
| `RZ` | Saved | | 1100 |
| `RT` | Argument/return/temporary | N/A | 1101 |
| `RSCRATCH0` | Assembler scratch | `RSCRATCH` | 1110 |
| `RSCRATCH1` | Assembler scratch | | 1111 |

The `R0` register MUST be hard-wired to ignore writes and always return the value `0b00000000` on reads. Wide registers are formed by two general-purpose registers, which MUST be equivalent to using the two 8-bit register names in order. For instance, `RAB` MUST be semantically equivalent to `RA, RB` in that order. General purpose registers are taken as the operands to most instructions which operate on registers.

Argument and return registers SHOULD be `RA - RF` and `RT`. The `RT` register SHOULD be used as the first return register, if the value being returned logically fits in a single (not wide) register. Otherwise, the register pair `RAB` SHOULD be preferred as the first return register. Argument registers SHOULD also be usable as temporary registers (caller-saved). `RU` through `RZ` SHOULD be saved registers (callee-saved).

The `RSCRATCH` wide register SHOULD be reserved as a "scratch" 16-bit register for the assembler. Certain pseudo-instructions and other macros MAY use this register when a temporary value is needed (like loading an address to jump to). As such, the value of `RSCRATCH` SHOULD not be assumed to persist over time.

## 1.3 Control and Status Registers

A few special registers MUST also be supported, which cannot be used as instruction operands, but can be read and written from general purpose registers with the `RSR` and `WSR` instructions. Special registers are generally wider 16-bit registers that are required for the operation of certain instructions.

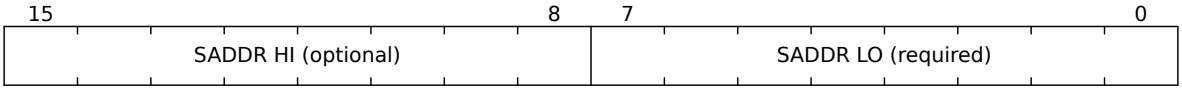
When used as operands for instructions, the special registers are encoded as follows:

| ABI Name | Encoding |
|---|---|
| `SADDR` | 0001 |
| `RADDR` | 0010 |
| `FLAGS` | 0011 |
| `CAUSE` | 0100 |
| `EXTNS` | 0101 |
| `RETSC` | 1000 (under System Call Extension) |
| `RETIR` | 1001 (under Interrupt Extension) |
| `PRIVS` | 1010 (under Privilege Extension) |

The following sub-sections describe the purpose and layout of each special register.

### 1.3.1 `SADDR`

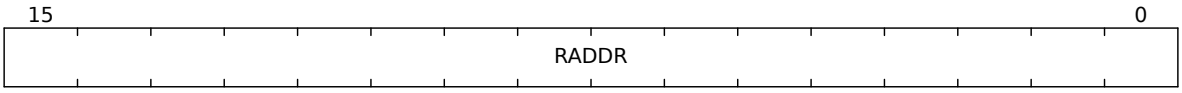
The stack address or stack pointer. `SADDR` MUST be at least an 8-bit register (supporting stack addresses `0x00` - `0xFF` with a possible high 8-bit prefix) and MUST not be more than a 16-bit register.




### 1.3.2 `RADDR`

The return address. `RADDR` MUST be wide enough to store an absolute address for the connected memory, which SHOULD be 16-bit.

The return address special register is not used by specific base instructions, but provides a dedicated 16-bit-wide register for subroutine calling. The `JAL` instruction MUST set the `RADDR` register with the address of the instruction after the `JAL`. Returning to the location in `RADDR` can be done with a `RSR` and `JMP` or a `JMP` using `RADDR` as the branch target.




### 1.3.3 FLAGS

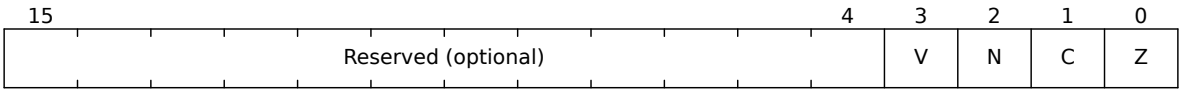
Conditional branching in the MISA architecture is based on ALU flag values for comparing integers. Both branching instructions, `JAL` and `JMP`, MUST these flags to make decisions on whether a branch will be taken.

`FLAGS` MUST be set based on the result of each ALU operation. The instructions which MUST update the `FLAGS` register are: `ADD`, `ADC`, `SUB`, `SBB`, `AND`, `OR`, `XOR`, `RRC`. Instructions which MUST NOT update the `FLAGS` register are: `HALT`, `SET`, `LD`, `ST`, `RSR`, `WSR` (MUST NOT update `FLAGS` asynchronously through the ALU output, but MUST update `FLAGS` if writing directly to it), `JAL`, `JMP`. A conditional branch that relies on `FLAGS` SHOULD be performed immediately after the conditional comparison or a `WSR` to `FLAGS`.

The `FLAGS` special register is a flags register (one bit per flag) which MUST be at least 4 bits and support the following:

| Flag | Bit | Commentary |
|---|---|---|
| `Z` | 0 | Whether the last ALU operation's output was zero. |
| `C` | 1 | The adder carry out bit of the last ALU operation. |
| `N` | 2 | Whether the last ALU operation's output was negative (MSB = 1). |
| `V` | 3 | Whether the last ALU operation's output overflowed (sign inversion). |

The `C` and `V` flags must always be `0` after executing `AND`, `OR`, `XOR`, or `RRC`.




The `FLAGS` register can be accessed indirectly with the conditional jump instructions using special assembler symbols:

| ABI Name | Logical Meaning | Encoding |
|---|---|---|
| `ALWAYS` | `1` | 000 |
| `EQUAL` | `Z` | 001 |
| `NOT_EQUAL` | `~Z` | 010 |
| `GREATER` | `~((N ^ V) \| Z)` | 011 |
| `LESS` | `N ^ V` | 100 |

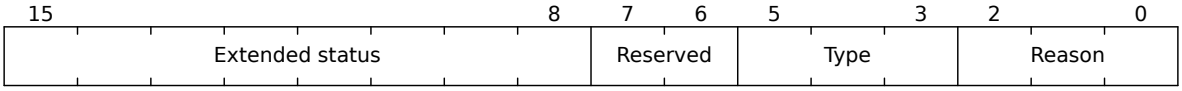
| ABI Name | Logical Meaning | Encoding |
|---|---|---|
| `GREATER_EQUAL` | `~(N ^ V)` | 101 |
| `LESS_EQUAL` | `(N ^ V) \| Z` | 110 |

These comparison flags, when passed as a conditional branch instruction operand, MUST perform the specified boolean logic on the fields of the `FLAGS` register. If the result is `1`, the branch is taken, otherwise the branch is not taken. The assumption is that a comparison of registers `RS1` and `RS2` is performed by invoking `SUB R0 RS1 RS2`, which sets the ALU flags appropriately for deciding `RS1 ? RS2` where `?` is a comparison operator.

### 1.3.4 `CAUSE`

The `CAUSE` special register holds information about the status of the processor during exceptions, faults, and termination. It MUST be set when the `HALT` instruction is executed, when syscalls are made under the System Call Extension, and when fault behavior (such as a malformed instruction decode) occurs during execution.

`CAUSE` is a 16-bit register with the following fields:




| Reason | Encoding |
|---|---|
| None | 000 |
| Instruction usage | 001 |
| Memory usage | 010 |
| Control event | 011 |

#### 1.3.4.1 `CAUSE` Reason: None

The `CAUSE` register does not represent an exception, or represents an unknown/generic exception without a reason. If the reason is 000, the Type and Extended status fields have no significance.

#### 1.3.4.2 `CAUSE` Reason: Instruction Usage

The exception cause is related to the use of a specific instruction or class of instructions. The valid Type fields are:

| Type | Encoding | Commentary |
|---|---|---|
| Halt instruction | 000 | The `HALT` instruction was executed. Extended status is the value of the `RS` operand register. Under the System Call Extension, this reason indicates that the `SYSCALL` instruction was executed. |
| Illegal instruction | 001 | An ill-formed instruction failed to decode. Extended status is not used. |
| Privileged instruction | 010 | A privileged instruction was executed while in User Mode (under Privilege Extension). |

#### 1.3.4.3 `CAUSE` Reason: Memory Usage

The exception cause is related to the use of an address or memory operation. The valid Type fields are:

| Type | Encoding | Commentary |
|---|---|---|
| Address overflow | 000 | A memory operation attempted to access an address which is outside the mapped memory space. |
| PC overflow | 001 | The program counter overflowed an unsigned 16-bit value. |
| Privileged access | 010 | An access to privileged memory was attempted while in User Mode (under Privilege Extension). |

#### 1.3.4.4 `CAUSE` Reason: Control Event

The exception cause is related to an exception or interrupt control flow event. The valid Type fields are:

| Type | Encoding | Commentary |
|---|---|---|
| Improper return context | 000 | Either: `JMP ALWAYS RETSC` was executed outside of a system call context or inside of an interrupt context; `JMP ALWAYS RETIR` was executed outside of an interrupt context. The extended status is set as follows: bit 0 = in system call context, bit 1 = in interrupt context, bits 2-3 = reserved, bits 4-7 = `JMP` target CSR register index. |
| Conditional context return | 001 | `JMP` was executed with a branch target of `RETSC` or `RETIR` but a comparison other than `ALWAYS`. The extended status is set as follows: bits 0-2 = comparison type, bit 3 = reserved, bits 4-7 = `JMP` target CSR register index. |

| Type | Encoding | Commentary |
|---|---|---|
| Improper system call | 010 | A system call was executed while the processor was in a system call or interrupt context. The extended status is set as follows: bit 0 = in system call context, bit 1 = in interrupt context. |

### 1.3.5 `EXTNS`

The `EXTNS` special register is a flags register that holds information about which (if any) architectural extensions are supported by the hardware. All implementations of the MISA architecture MUST include the `EXTNS` register, even those which only implement the core architecture. A core-only implementation can be identified by `EXTNS == 0x0000`.

Unlike most special registers, writes to `EXTNS` MAY be ignored (i.e. the register MAY have a hard-coded value). For more information on writes to `EXTNS`, see the Dynamic Hardware Extension. Note that bit 0 (the Dynamic Hardware Extension) MUST ignore writes regardless of its state, preventing the Dynamic Hardware Extension from disabling itself.

`EXTNS` MUST support at least the following bits:

| Bit | Extension |
|---|---|
| 0 | Dynamic Hardware Extension |
| 1 | System Call Extension |
| 2 | Privilege Extension |
| 3 | Interrupt Extension |

## 1.4 Data Formats and Alignments

The MISA architecture defined in this chapter only supports 8-bit signed integers using two's complement encoding, where the value of an integer is given by the sum of its active bits:

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| -128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |

16-bit data types MAY be supported in software using the `ADC` and `SBB` instructions for arithmetic.

## 1.5 Memory

The MISA architecture MAY support up to a 16-bit address bus for memory operations, and memory MUST be byte-addressable. While the memory map for applications using the MISA architecture is generally free for the user to decide, this section will cover both a recommended map and specific regions which are required by all maps.

### 1.5.1 Required Memory Regions

The region 0xFFF0 - 0xFFFF MUST be reserved for a reset vector table. When the processor is taken out of reset, the program counter MUST be initialized to the address stored in memory at 0xFFFE - 0xFFFF and begin execution there.

The reset vector table contains the following function pointers:

| Vector Address | Purpose |
|---|---|
| `0xFFF0 - 0xFFF1` | Reserved |
| `0xFFF2 - 0xFFF3` | Reserved |
| `0xFFF4 - 0xFFF5` | Reserved |
| `0xFFF6 - 0xFFF7` | Reserved |
| `0xFFF8 - 0xFFF9` | Interrupt handler (under Interrupt Extension) |
| `0xFFFA - 0xFFFB` | Syscall handler (under System Call Extension) |
| `0xFFFC - 0xFFFD` | Fault handler |
| `0xFFFE - 0xFFFF` | Reset handler |

### 1.5.2 Recommended Memory Map

The linker script recommended by default is as follows:

```
// Zero page, which may have fast access on some hardware
SECTION 0x0000 - 0x00FF : direct ;

// Stack, SADDR initialized to 0x(01)FF and decrements
SECTION 0x0100 - 0x01FF : ;

// RAM, may support software-implemented stack or heap
SECTION 0x0200 - 0x7FFF : data bss ;

// Memory-mapped IO and IO buffers
SECTION 0x8000 - 0xBFFF : ;
```

```
// Program ROM
SECTION 0xC000 - 0xFFEF : text rodata ;

// Vector table
SECTION 0xFFF0 - 0xFFFF : vectors ;
```

Which defines the following sections by convention:

| Section | Commentary |
|---|---|
| `direct` | Contents to place in the first 256-byte page. This page may support faster memory operations on certain hardware implementations, and thus may be a desirable target for frequently accessed data. |
| `data` | Initialized global data that is mutable. |
| `bss` | Declared but uninitialized global data that is mutable. Runtime/startup code should initialize `bss` symbols to zero. |
| `text` | Executable and immutable program code. |
| `rodata` | Initialized global constants. |
| `vectors` | A small region for reset vector function addresses. |

### 1.5.3 Stack Conventions

The stack SHOULD be towards-lower-address, which grows towards `0x0000` on pushes and contracts towards `0xFFFF` on pops. This convention SHOULD be respected regardless of the supported width of the `SADDR` special register or the location in memory of the stack.

Stack operations SHOULD respect the convention of the `PUSH` and `POP` assembler macros. That is, pushing to the stack SHOULD write before decrementing `SADDR`, and popping from the stack SHOULD increment `SADDR` before reading.

Multi-byte pushes and pops SHOULD be logically equivalent to chaining several `PUSH` or `POP` assembler macros.

## 1.6 Reset

When the processor is taken out of the reset state, some internal state MUST be initialized:

- The reset vector address stored in memory at `0xFFFE - 0xFFFF` MUST be read and used to initialize the program counter.

The state of general purpose registers and control and status registers is not defined.

## 1.7 Faults

Fault conditions occur when something goes wrong during program execution. This includes:

- Bad instruction decodes
- Overflow of the program counter
- Etc.

When a fault is detected, the processor MUST set the `CAUSE` register based on the type of fault and then set the program counter to the fault vector address stored in memory at `0xFFFC - 0xFFFD`. Execution MUST then continue in the fault handler, which is software-defined and MAY attempt recovery, dump diagnostic information, or simply halt.

## 1.8 Exceptions and Interrupts

The core ISA defined in this chapter does not support traditional exceptions or interrupts. The `CAUSE` special register is intended for use as a diagnostic tool in the core architecture, and `HALT` is the only instruction that MUST update it.

A basic synchronous exception system is supported under the System Call Extension which, when combined with the Privilege Extension, MAY be used to invoke privileged routines from the unprivileged user mode.

The Interrupt Extension provides support for basic asynchronous interrupts, which MAY be used to let the processor more easily keep time and interact with peripherals.

## 1.9 Base Instructions

Sixteen base instructions (with distinct opcodes and behavior) MUST be supported by the MISA architecture. This is a hard limit, and the 8-bit MISA architecture does not allow for more than 4 bits of opcode in each instruction word. This subsection defines all the base instructions and their behavior on the microprocessor's state.

Instruction behavior is given under the "Meaning" label, which is a formalism for how the instruction should affect the behavior of the processor. The following syntax is used when describing instruction behavior:

- `Register`: The general-purpose register file, indexed by the register number from 0-15
- `Special`: The special register file, indexed by the special register number from 0-15
- `Memory`: The mapped memory, indexed by an address from 0-64K
- `Check`: The truth value from the current state of `FLAGS`, indexed by a comparison operand
- `PC`: The program counter

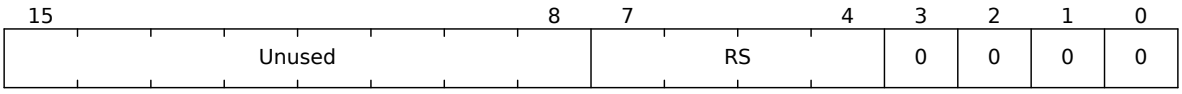
- `Halt`: Stop processor execution
- `Update`: Update a value using a definition/ruleset provided elsewhere in this chapter
- `SignExtN`: Copy the MSB of a narrow field to all bits added when converting to an `N`-bit field
- `If`: Perform the encapsulated operations only if a condition is true

### 1.9.1 `HALT`

Usage: `HALT RS`

Description: Halt the processor and emit the contents of `RS` for debug.

Encoding:




Meaning:

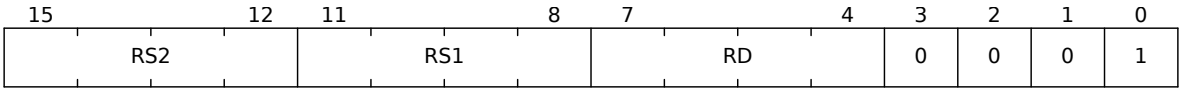
```
Special[CAUSE] = Register[RS] << 8 | 0x01;
Halt;
```

### 1.9.2 `ADD`

Usage: `ADD RD RS1 RS2`

Description: Add the content of `RS1` and `RS2` and store the result in `RD`.

Encoding:




Meaning:

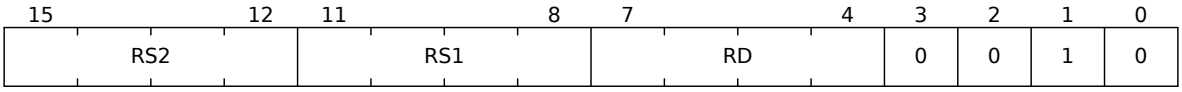
```
Register[RD] = Register[RS1] + Register[RS2];
Update Special[FLAGS];
```

### 1.9.3 ADC

Usage: `ADC RD RS1 RS2`

Description: Add the contents of `RS1`, `RS2`, and the `C` field of the `FLAGS` register and store the result in `RD`.

Encoding:




Meaning:

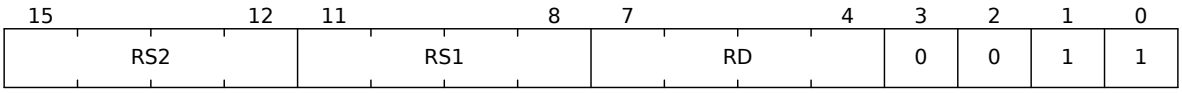
```
Register[RD] = Register[RS1] + Register[RS2] + Special[FLAGS.C];
Update Special[FLAGS];
```

### 1.9.4 SUB

Usage: `SUB RD RS1 RS2`

Description: Subtract the contents of `RS2` from `RS1` and store the result in `RD`.

Encoding:




Meaning:

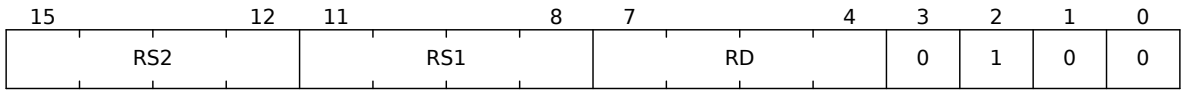
```
Register[RD] = Register[RS1] - Register[RS2];
Update Special[FLAGS];
```

### 1.9.5 SBB

Usage: `SBB RD RS1 RS2`

Description: Add the contents of `RS1`, the bitwise complement of `RS2`, and the `C` field of the `FLAGS` register, and store the result in `RD`. Because `C` is the adder carry out, this equals `RS1 - RS2 - (1 - C)`, so chaining `SUB` then `SBB` subtracts a multi-byte value with the borrow carried through `C`.

Encoding:




Meaning:

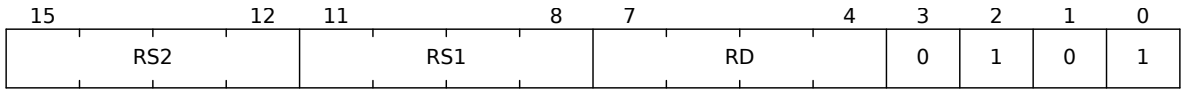
```
Regsiter[RD] = Register[RS1] + ~Register[RS2] + Special[FLAGS.C];
Update Special[FLAGS];
```

### 1.9.6 AND

Usage: `AND RD RS1 RS2`

Description: Compute the bitwise AND of the contents of `RS1` and `RS2` and store the result in `RD`.

Encoding:




Meaning:

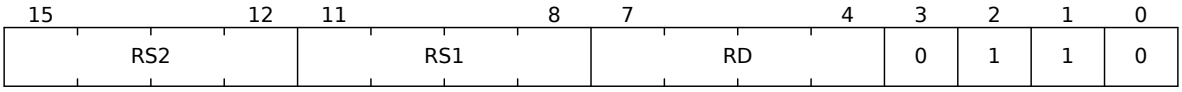
```
Register[RD] = Register[RS1] & Register[RS2];
Update Special[FLAGS];
```

### 1.9.7 OR

Usage: `OR RD RS1 RS2`

Description: Compute the bitwise OR of the contents of `RS1` and `RS2` and store the result in `RD`.

Encoding:




Meaning:

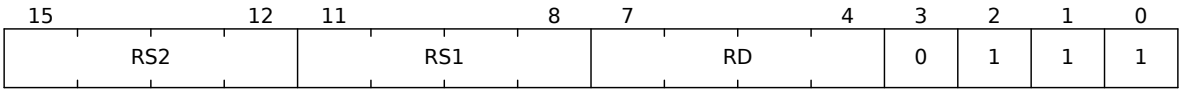
```
Regsiter[RD] = Register[RS1] | Register[RS2];
Update Special[FLAGS];
```

### 1.9.8 XOR

Usage: `XOR RD RS1 RS2`

Description: Compute the bitwise exclusive OR of the contents of `RS1` and `RS2` and store the result in `RD`.

Encoding:




Meaning:

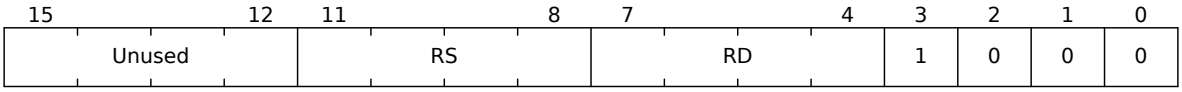
```
Regsiter[RD] = Register[RS1] ^ Register[RS2];
Update Special[FLAGS];
```

### 1.9.9 RRC

Usage: `RRC RD RS`

Description: Shift the 9-bit integer formed by `{C, RS}` right by one bit. Store the shifted result in `RD` and the LSB of `RS` into `C`. The ALU flags are updated from the 8-bit result. `Z` is set when the result is zero, `N` takes the most significant bit of the result, and `V` is cleared. `C` receives the shifted-out LSB of `RS` as described above.

Encoding:




Meaning:

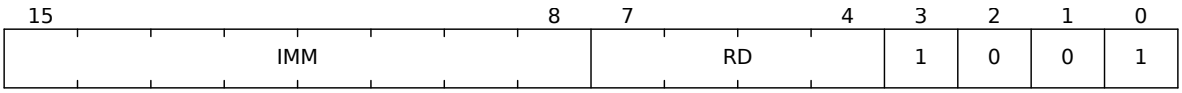
```
Register[RD] = (Special[FLAGS.C] << 0x07) | (Register[RS] >> 0x01);
Update Special[FLAGS];
Special[FLAGS.C] = Register[RS] & 0x01;
```

### 1.9.10 SET

Usage: `SET RD IMM`

Description: Store the provided 8-bit immediate value into `RD`.

Encoding:




Meaning:

```
Register[RD] = IMM;
```

### 1.9.11 LD

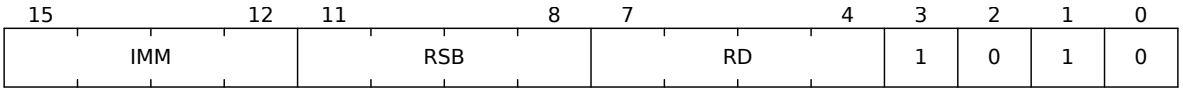
Usage: `LD RD RSB IMM`

Description: Load the 8-bit word at the address formed by `{RSB, RSB + 1} + IMM` into `RD`. The base address is stored in two adjacent general purpose registers; the instruction names the high register `RSB`, and the low byte is taken from register `RSB + 1`.

The encoding `0xF` for the base register `RSB` is reserved for stack operations and has a separate behavior.

The immediate operand is a 4-bit two's complement integer ranging from `-8` to `7`. It specifies a byte offset to add to the base address in the general purpose register pair.

Encoding:




Meaning:

```
If (RSB != 0xF) {
  Register[RD] = Memory[((Register[RSB] << 0x08) | Register[RSB + 1]) + SignExt16 IMM];
}
If (RSB == 0xF) {
  Special[SADDR] = Special[SADDR] + SignExt16 IMM;
  Register[RD] = Memory[Special[SADDR]];
}
```

### 1.9.12 ST

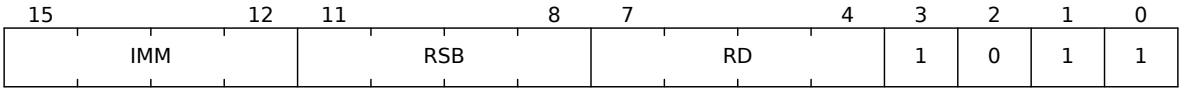
Usage: `ST RD RSB IMM`

Description: Store the 8-bit word in `RD` to the address formed by `{RSB, RSB + 1} + IMM`. The base address is stored in two adjacent general purpose registers; the instruction names the high register `RSB`, and the low byte is taken from register `RSB + 1`.

The encoding `0xF` for the base register `RSB` is reserved for stack operations and has a separate behavior.

The immediate operand is a 4-bit two's complement integer ranging from `-8` to `7`. It specifies a byte offset to add to the base address in the general purpose register pair.

Encoding:




Meaning:

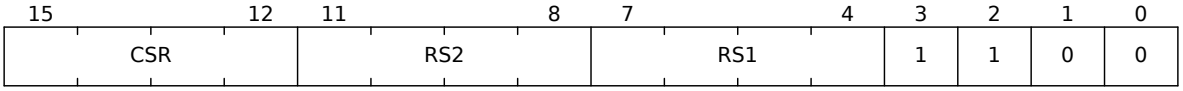
```
If (RSB != 0xF) {
  Memory[((Register[RSB] << 0x08) | Register[RSB + 1]) + SignExt16 IMM] = Register[RD];
}
If (RSB == 0xF) {
  Memory[Special[SADDR]] = Register[RD];
  Special[SADDR] = Special[SADDR] + SignExt16 IMM;
}
```

### 1.9.13 RSR

Usage: `RSR CSR RS1 RS2`

Description: Read the 16-bit control and status register `CSR` into the two 8-bit general purpose registers `{RS1, RS2}`

Encoding:




Meaning:

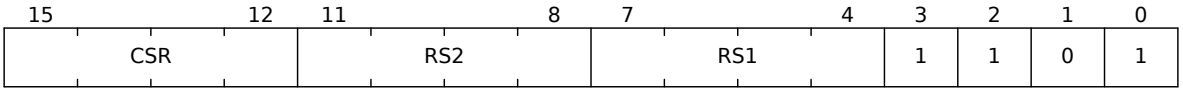
```
Register[RS1] = Special[CSR] >> 0x08;
Register[RS2] = Special[CSR] & 0xFF;
```

### 1.9.14 WSR

Usage: `WSR CSR RS1 RS2`

Description: Write the contents formed by `{RS1, RS2}` into the 16-bit control and status register `CSR`.

Encoding:




Meaning:

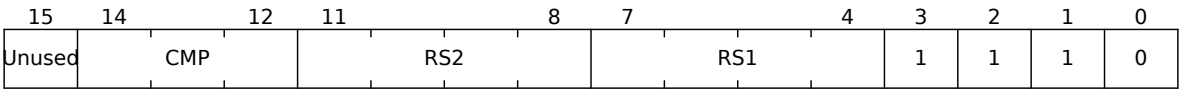
```
Special[CSR] = (Register[RS1] << 0x08) | Register[RS2];
```

### 1.9.15 JAL

Usage: `JAL CMP RS1 RS2`

Description: If the comparison type `CMP` is true as determined by the current ALU flags, set `RADDR` to the address of the next instruction and set `PC` to the address formed by `{RS1, RS2}`. Note that `JAL` does not support using special registers as a branch target like `JMP` does.

Encoding:




Meaning:

```
If (Check[CMP]) {
  Special[RADDR] = PC + 0x02;
  PC = (Register[RS1] << 0x08) | Register[RS2];
}
```

### 1.9.16 JMP

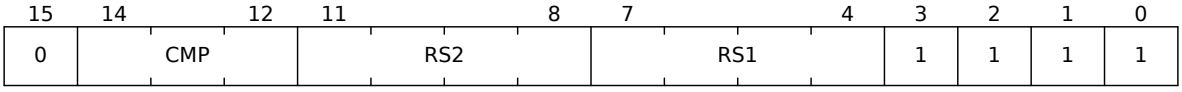
`JMP` has two variants depending on the branch target. The most significant bit of the instruction encoding identifies the target source.

#### 1.9.16.1 General Purpose Register Target

Usage: `JMP CMP RS1 RS2`

Description: If the comparison type `CMP` is true as determined by the current ALU flags, set `PC` to the address formed by `{RS1, RS2}`.

Encoding (when the branch target is stored in two general-purpose registers):




Meaning:

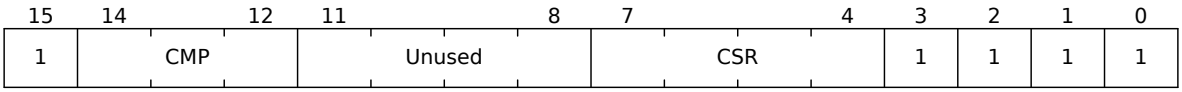
```
If (Check[CMP]) {
  PC = (Register[RS1] << 0x08) | Register[RS2];
}
```

#### 1.9.16.2 Special Register Target

Usage: `JMP CMP CSR`

Description: If the comparison type `CMP` is true as determined by the current ALU flags, set `PC` to the address in `CSR`. Jumping to the 16-bit value in a special register is intended to be used for quickly returning to addresses like `RADDR`, `RETSC`, and `RETIR`.

Encoding (when the branch target is stored in one special register):




Meaning:

```
If (Check[CMP]) {
  PC = Special[CSR];
}
```

## 1.10 Pseudo Instructions

Because of the limitations of the core instruction set, implementations SHOULD additionally provide the following set of assembler macros whose functionality can be represented by one or more of the base instructions.

The semantic meaning of pseudo instructions is given in terms of a sequence of base instructions to be executed in order. Implementations providing pseudo instructions MAY use a different definition, but the semantic behavior MUST match.

### 1.10.1 `ADD2`

Usage: `ADD2 RD1 RD2 RS1 RS2 RS3 RS4`

Description: Add the 16-bit values `{RS1, RS2}` and `{RS3, RS4}` and store the result in `{RD1, RD2}`.

Meaning:

```
ADD RD2 RS2 RS4
ADC RD1 RS1 RS3
```

### 1.10.2 `AND2`

Usage: `AND2 RD1 RD2 RS1 RS2 RS3 RS4`

Description: Compute the bitwise AND of the contents of `{RS1, RS2}` and `{RS3, RS4}` and store the result in `{RD1, RD2}`

Meaning:

```
AND RD2 RS2 RS4
AND RD1 RS1 RS3
```

### 1.10.3 CALL

Usage: `CALL IMM`

Description: Jump to the subroutine at the address `IMM` and link with the `RADDR` special register.

Meaning:

```
SET RSCRATCH0 (IMM >> 0x08)
SET RSCRATCH1 (IMM & 0xFF)
JAL ALWAYS RSCRATCH0 RSCRATCH1
```

### 1.10.4 CLR

Usage: `CLR`

Description: Set the `FLAGS` special register to zero, effectively lowering all flags.

Meaning:

```
WSR FLAGS R0 R0
```

### 1.10.5 CMP

Usage: `CMP RS1 RS2`

Description: Perform an arithmetic operation which will set the `FLAGS` register such that the boolean operation `?` can be performed on `RS1 ? RS2`.

Meaning:

```
SUB R0 RS1 RS2
```

### 1.10.6 GOTO

Usage: `GOTO IMM`

Description: Unconditionally jump to the address `IMM`.

Meaning:

```
SET RSCRATCH0 (IMM >> 0x08)
SET RSCRATCH1 (IMM & 0xFF)
JMP ALWAYS RSCRATCH0 RSCRATCH1
```

### 1.10.7 JALI

Usage: `JALI CMP IMM`

Description: If the comparison type `CMP` is true as determined by the current ALU flags, set `RADDR` to the address of the next instruction and set `PC` to the address `IMM`.

Meaning:

```
SET RSCRATCH0 (IMM >> 0x08)
SET RSCRATCH1 (IMM & 0xFF)
JAL CMP RSCRATCH0 RSCRATCH1
```

### 1.10.8 JMPI

Usage: `JMPI CMP IMM`

Description: If the comparison type `CMP` is true as determined by the current ALU flags, set `PC` to the address `IMM`.

Meaning:

```
SET RSCRATCH0 (IMM >> 0x08)
SET RSCRATCH1 (IMM & 0xFF)
JMP CMP RSCRATCH0 RSCRATCH1
```

### 1.10.9 MOV

Usage: `MOV RD RS1`

Description: Copy the contents of `RS1` into `RD`.

Meaning:

```
OR RD RS1 R0
```

### 1.10.10 MOV2

Usage: `MOV2 RD1 RD2 RS1 RS2`

Description: Copy the contents of the 16-bit value `{RS1, RS2}` into `{RD1, RD2}`.

Meaning:

```
MOV RD2 RS2
MOV RD1 RS1
```

### 1.10.11 NOP

Usage: `NOP`

Description: Do nothing.

Meaning:

```
SET R0 0x00
```

### 1.10.12 OR2

Usage: `OR2 RD1 RD2 RS1 RS2 RS3 RS4`

Description: Compute the bitwise OR of the contents of `{RS1, RS2}` and `{RS3, RS4}` and store the result in `{RD1, RD2}`.

Meaning:

```
OR RD2 RS2 RS4
OR RD1 RS1 RS3
```

### 1.10.13 POP

Usage: `POP RD`

Meaning:

```
LD RD RSCRATCH1 +1
```

Description: Increment `SADDR` by 1 byte and then load the byte at the new stack address into `RD`.

### 1.10.14 POP2

Usage: `POP2 RD1 RD2`

Description: Pop a 16-bit value off the stack and into `{RD1, RD2}`.

Meaning:

```
LD RD1 RSCRATCH1 +1
LD RD2 RSCRATCH1 +1
```

### 1.10.15 PUSH

Usage: `PUSH RS`

Meaning:

```
ST RS RSCRATCH1 -1
```

Description: Write `RS` onto the stack at `SADDR`, and then decrement `SADDR` by 1 byte.

### 1.10.16 PUSH2

Usage: `PUSH2 RS1 RS2`

Description: Push a 16-bit value onto the stack from `{RS1, RS2}`.

Meaning:

```
ST RS2 RSCRATCH1 -1
ST RS1 RSCRATCH1 -1
```

### 1.10.17 RET

Usage: `RET`

Description: Return to the address in `RADDR`.

Meaning:

```
JMP ALWAYS RADDR
```

### 1.10.18 RRC2

Usage: `RRC2 RD1 RD2 RS1 RS2`

Description: Shift the 17-bit integer formed by `{C, RS1, RS2}` right by one bit. Store the shifted result in `{RD1, RD2}` and the LSB of `RS2` into `C`.

Meaning:

```
RRC RD1 RS1
RRC RD2 RS2
```

### 1.10.19 SET2

Usage: `SET2 RS1 RS2 IMM`

Description: Set both the `RS1` and `RS2` registers from the high and low halves of a 16-bit immediate value.

Meaning:

```
SET RS1 (IMM >> 0x8)
SET RS2 (IMM & 0xFF)
```

### 1.10.20 SUB2

Usage: `SUB2 RD1 RD2 RS1 RS2 RS3 RS4`

Description: Subtract the 16-bit value `{RS3, RS4}` from `{RS1, RS2}` and store the result in `{RD1, RD2}`.

Meaning:

```
SUB RD2 RS2 RS4
SBB RD1 RS1 RS3
```

### 1.10.21 XOR2

Usage: `XOR2 RD1 RD2 RS1 RS2 RS3 RS4`

Description: Compute the bitwise XOR of the contents of `{RS1, RS2}` and `{RS3, RS4}` and store the result in `{RD1, RD2}`

Meaning:

```
XOR RD2 RS2 RS4
XOR RD1 RS1 RS3
```

## 1.11 Architecture Extensions

The content in this chapter until this point is the MISA core architecture. This section provides a description of several optional extensions to that architecture which enable additional functionalities.

### 1.11.1 Dynamic Hardware Extension

The Dynamic Hardware Extension enables writes to the `EXTNS` special register and thus dynamic support for the architecture extensions discussed in this section.

**Note:** If the Dynamic Hardware Extension is supported, writes to the Dynamic Hardware Extension bit of the `EXTNS` special register MUST always be ignored. The extension cannot be used to disable itself at runtime.

If the Dynamic Hardware Extension is not supported, writes to any bit of `EXTNS` MUST be ignored. The list of supported extensions is static and a property of the hardware itself. If supported, writes MUST take effect on the instruction following the `WSR`.

Partial support of the Dynamic Hardware Extension is also allowed. That is, an implementation MAY only permit enabling/disabling specific, but not every extension at runtime. In that case, writes to the bit of any extension that cannot be toggled MUST be ignored. Software should treat the lack of change in the target bit as a signal that the target extension is permanently enabled/disabled.

### 1.11.2 System Call Extension

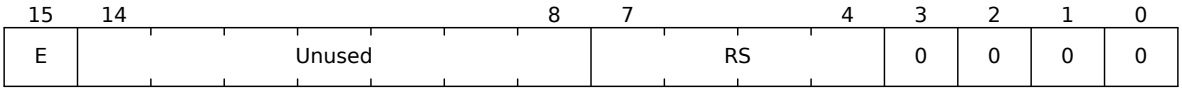
The System Call Extension extends the `HALT` instruction in the core architecture to support basic synchronous exceptions that MAY be used for system calls and other similar features.

#### 1.11.2.1 SYSCALL

The SYSCALL instruction replaces the HALT instruction in the assembler under the System Call Extension. Its usage is similar to HALT, but with a conditional "exception" bit that controls the terminal behavior of the processor.

Usage: SYSCALL RS

Encoding:




Meaning:

```
Special[CAUSE] = Register[RS] << 8 | 0x01;
If (E == 0b0) {
  Halt;
}
If (E == 0b1) {
  PC = (Memory[0xFFFB] << 0x08) | Memory[0xFFFA];
}
```

#### 1.11.2.2 HALT

HALT functionality as defined in the core architecture is supported under the System Call Extension by setting the E bit to 0. Assemblers MUST set E to 0 when the HALT opcode is used, and set E to 1 when the SYSCALL opcode is used.

#### 1.11.2.3 Calling Conventions and Stack Frame

When a SYSCALL instruction is executed, the contents of the RS register are used as the syscall code, stored in the CAUSE special register. The syscall handler address at 0xFFFA - 0xFFFB is responsible for reading the CAUSE register, understanding that a syscall was executed, and using the extended status value to branch to different handler code for each type of call.

General purpose registers follow the same caller-callee conventions as standard subroutine/function calls in the MISA ABI. This includes which registers are used as arguments to the system call, and which registers are used as return values from the system call. All special registers MUST be saved and restored by the system call routine.

#### 1.11.2.4 Nested System Calls

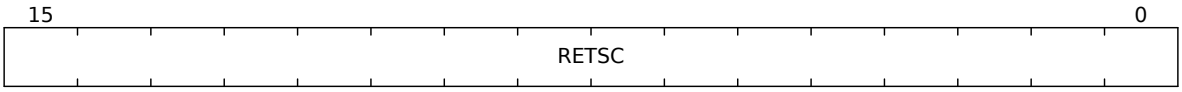
The `SYSCALL` instruction cannot be executed within a system call context. The processor is canonically in a system call context under the following formal definition:

- The processor enters a system call context when the `SYSCALL` instruction is executed with the exception bit set
- The processor leaves the system call context when a `JMP ALWAYS RETSC` instruction is executed
- Every instruction executed between entering and leaving the system call context exists within the system call context

If a secondary `SYSCALL` is performed in a system call context, a fault MUST occur with the `CAUSE` Reason "Control event" and the `CAUSE` Type "Improper system call".

#### 1.11.2.5 Returning From System Calls

To return from system calls, the `RETSC` special register is defined, which stores the return address when a `SYSCALL` instruction is executed.




The following pseudo-instruction is defined to return to this address easily:

Usage: `RETS`

Description: Return from a system call to the address in `RETSC`.

Meaning:

```
JMP ALWAYS RETSC
```

### 1.11.3 Interrupt Extension

The Interrupt Extension adds support for basic asynchronous, non-deterministic exceptions. While the synchronous exceptions in the System Call Extension come from code being executed by the processor, interrupts are triggered externally to allow the processor to keep time, communicate with peripherals, and more.

#### 1.11.3.1 Triggering Interrupts

MISA processors supporting the interrupt extension MUST designate one pin as the interrupt pin. The interrupt pin MAY be active-high or active-low. The interrupt pin is set to its active state to trigger an interrupt – the rising or falling edge of the pin MUST cause the processor to finish executing the current instruction and then set the program counter to the interrupt handler address at `0xFFF8 - 0xFFF9`.

When this happens, interrupts MUST be automatically disabled by the processor. If the interrupt pin is toggled, another interrupt MUST not be detected. Taking interrupts does not affect the `CAUSE` register like system calls or faults.

#### 1.11.3.2 Interrupt Queue

Information about which interrupt was triggered SHOULD be provided to the processor through a memory-mapped queue. The queue MAY be located anywhere in memory.

The queue SHOULD be structured as follows:

| Bytes | Purpose |
|---|---|
| 0 | Length |
| 1 | Pop signal |
| 2 | Pop acknowledgement |
| 3 | Interrupt number |
| 4-5 | Argument pointer |

Every field except for the pop signal MUST only be read by the processor. Writing to other fields is undefined behavior.

The length field defines the number of outstanding interrupts. The interrupt handler SHOULD respond to every interrupt in the queue before returning, as it is not guaranteed that the interrupt source will reassert the interrupt pin. If the queue is not emptied before returning, interrupts will be left unaddressed until the next time the pin is asserted.

When responding to an interrupt, the interrupt handler should read the number and argument pointer fields, and then pop the entry using the pop handshake.

The pop handshake uses the pop signal and pop acknowledgement fields as request and acknowledge signals. The pop signal is written by the processor to any non-zero value when the current interrupt should be popped from the queue. The pop acknowledgement is written by the external hardware to any non-zero value when the requested pop is completed and the queue fields are updated. Both fields are zero in the idle state, including after reset and when the Interrupt Extension is first enabled.

The handshake proceeds in four phases. To pop an entry, the processor MUST:

1. Write a non-zero value to the pop signal to request the pop.
2. Wait until the pop acknowledgement is non-zero.
3. Write zero to the pop signal to withdraw the request.
4. Wait until the pop acknowledgement is zero before beginning another pop.

In response, the external hardware must:

1. On the pop signal becoming non-zero, remove the head entry, update the length and (if the queue is not empty) the number and argument pointer fields to reflect the new head, and only then write a non-zero value to the pop acknowledgement.
2. On the pop signal returning to zero, write zero to the pop acknowledgement.

While the handshake is idle (both the pop signal and pop acknowledgement are zero) and the length is non-zero, the number and argument pointer fields MUST hold the head entry. To preserve this, the external hardware MUST modify those fields only during a pop (between observing the pop signal non-zero and asserting the pop acknowledgement) or, when enqueuing into an empty queue, before incrementing the length. The processor MUST read those fields only while the handshake is idle and the length is non-zero.

The exact use of the interrupt number and argument pointer fields are implementation-defined. The intent is for the interrupt handler to dispatch to a specific service routine based on the number, passing the argument pointer as a pointer to a particular type of structure containing information for the service routine. When the length of the queue is zero, the values of the interrupt number and argument pointer are undefined.

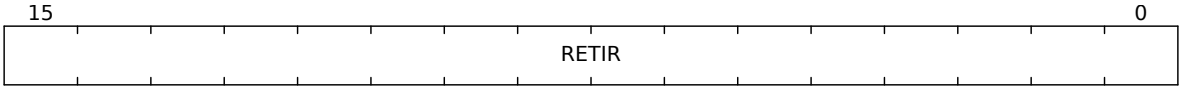
#### 1.11.3.3 Calling Conventions

Because interrupts are not necessarily triggered by the running code in the way system calls are, the entire processor state MUST be considered to be callee-saved in an interrupt context. The interrupt handler is responsible for saving and restoring every special and general purpose register that it modifies.

Three exceptions are the `RSCRATCH0` and `RSCRATCH1` general-purpose registers, and the `FLAGS` special register. These registers MUST be automatically saved by the processor when an interrupt is triggered and automatically restored when returning from the interrupt. On entry, `RSCRATCH` is therefore the only register the handler can freely use before saving anything.

#### 1.11.3.4 Returning From Interrupts

To return from interrupts, the `RETIR` special register is defined, which stores the return address when an interrupt is triggered.




The following pseudo-instruction is defined to return to this address easily:

Usage: `RETI`

Description: Return from an interrupt to the address in `RETIR`.

Meaning:

```
JMP ALWAYS RETIR
```

When the processor executes `JMP ALWAYS RETIR`, it MUST also perform the following actions atomically:

1. Restore `RSCRATCH0`, `RSCRATCH1`, `FLAGS`, and `PRIVS.R` (under the Privilege Extension)
2. Set the program counter to the address in `RETIR`
3. Leave the interrupt context and reenable interrupts

Software MUST not return from an interrupt via `RSR RETIR RSCRATCH; JMP ALWAYS RSCRATCH`, as this is not guaranteed to complete steps 1 and 3.

#### 1.11.3.5 Masking Interrupts

Interrupt masking MUST not be controlled by software. If the Interrupt Extension is supported, interrupts can always be triggered. Only two conditions prevent the processor from taking new interrupts:

1. Disabling the Interrupt Extension through the Dynamic Hardware Extension.
2. Being in an interrupt context.

The processor is canonically in an interrupt context under the following formal definition:

- The processor enters an interrupt context when the program counter is set to the address of the interrupt handler as a result of the interrupt pin going active
- The processor leaves an interrupt context when a `JMP ALWAYS RETIR` instruction is executed

- Every instruction executed between entering and leaving the interrupt context exists within the interrupt context

Software MAY choose to ignore the interrupt by returning immediately from the interrupt handler.

#### 1.11.3.6 System Calls in Interrupts

While the processor is in an interrupt context, system calls SHOULD not be executed. If a system call is performed in an interrupt context, the processor MUST fault with the reason "Control event" and the type "Improper system call".

#### 1.11.3.7 Canonical Interrupt Handler

With `RSCRATCH` and `FLAGS` saved by the processor, interrupt handlers are free to modify them. One method of saving and restoring the full context in the interrupt handler is as follows:

```
Interrupt_Handler:
    PUSH2 RAB
    PUSH2 RCD
    ...                         ; Save other general purpose registers

    RSR RADDR RAB
    PUSH2 RAB
    ...                         ; Save other special registers

    ;; Read the interrupt queue, dispatch a service routine, drain the queue

    ...                         ; Restore other special registers
    POP2 RAB
    WSR RADDR RAB

    ...                         ; Restore other general purpose registers
    POP2 RCD
    POP2 RAB

    RETI                        ; Return from the interrupt
```

### 1.11.4 Privilege Extension

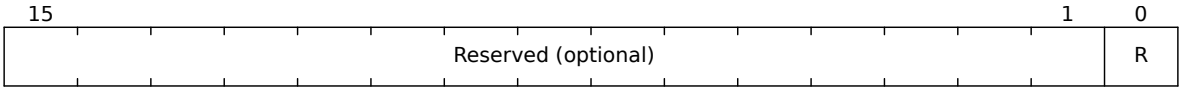
The Privilege Extension segments the capabilities of the processor into two distinct levels of privilege, which MAY be used to limit the actions taken by untrusted software.

Machine Mode grants access to all functionality of the processor, and is similar in scope to the core architecture without the Privilege Extension. User Mode restricts access to certain features of the core architecture, and provides a way to manage memory I/O at a more cautious level of privilege.

#### 1.11.4.1 `PRIVS`

Privilege is tracked and controlled using the `PRIVS` special register, which is defined as part of the Privilege Extension. It is a flags register with the following fields:

| Flag | Bit | Commentary |
|---|---|---|
| `R` | 0 | The privilege ring bit. 0 for Machine Mode, 1 for User Mode. |




#### 1.11.4.2 User Mode Restrictions

When the processor is in User Mode, the following actions MUST cause a privilege-related fault:

- Execution of `HALT`
- Execution of `RSR` except on `SADDR`, `RADDR`, `FLAGS`, and `PRIVS`
- Execution of `WSR` except on `SADDR`, `RADDR`, and `FLAGS`
- Access to privileged memory, if a memory protection unit is used to manage access based on the ring bit

If a privilege error is detected, the processor MUST preempt the instruction which caused the error, discard any partial result of the instruction, and jump to the fault handler. The `CAUSE` special register MUST be set to indicate the privilege error for the instruction use or memory access.

#### 1.11.4.3 Points of Privilege Change

Under normal operation, the processor is expected to change privilege modes when the following occur:

- When any fault occurs which requires the processor to jump to the fault handler (including an User Mode privilege error), the processor must first set `PRIVS.R` to 0.
- Under the System Call Extension, when the `SYSCALL` instruction is executed and the exception bit is 1, `PRIVS.R` must be set to 0 before the system call handler is entered.
- Under the System Call Extension, when the processor is returned from a system call context, the `PRIVS.R` bit must be restored to its initial value before the `SYSCALL`.
- Under the Interrupt Extension, when an interrupt is taken, `PRIVS.R` must be set to 0 before the interrupt handler is entered.
- Under the Interrupt Extension, when the processor is returned from an interrupt context, the `PRIVS.R` bit must be restored to its initial value before the interrupt occurred. Note that interrupts may trigger while the processor is in a system call context, in which case returning from the interrupt and returning from the system call will restore different ring bit values.
- When `WSR` is executed on the `PRIVS` special register from Machine Mode, the updated ring bit must take effect for the next instruction.

All privilege changes MUST be performed automatically by the processor when an appropriate condition is detected, except for direct writes to `PRIVS` via `WSR` from Machine Mode.

# 2 Toolchain

The MISA project provides a standard toolchain for low-level development of programs on a processor supporting the full MISA architecture. This includes an assembler, linker, some debug/analysis tools, and a reference simulator.

This chapter covers the usage and technical details of the toolchain for building programs. That is, all components of the MISA software package except the functional simulator.

## 2.1 Assembler

`misa-as` in the reference assembler for the MISA instruction set architecture described in Chapter 1 of this document. It supports all the behavior of the ISA chapter, including the recommended pseudo instructions and assembler macros.

Using `misa-as` is similar to an abbreviated version of the GNU `as` assemblers and others like it. For a full description, see the help text at `misa-as --help`. In short, there are two common ways to use the assembler:

1) Assembling source files to a binary: `misa-as file1.S file2.S ... fileN.S`
2) Assembling source files to object files: `misa-as -a file1.S file2.S ... fileN.S`

The assembler produces object files in a simple format accepted by the MISA linker. By default, assembling will also invoke the linker to produce a final binary, although this step can be omitted.

### 2.1.1 Assembler Pipeline

The `misa-as` assembler is a wrapper CLI utility for several core binaries including the assembler, but also the preprocessor and linker.

Assembling with no early-exit flags (e.g. `-p` or `-a`) runs provided files through a multi-step pipeline:

1) All assembly source files are passed to the MISA preprocessor to resolve any preprocessor directives. The pipeline can be stopped after this stage with the `-p` flag.

2) The preprocessed output files are passed to the MISA assembler, which produces object files. The pipeline can be stopped after this stage with the `-a` flag.
3) The generated object files, as well as any non-source files originally passed to `misa-as`, are passed to the MISA linker to produce the final flat binary.

### 2.1.2 Assembler Command Line

`misa-as` classifies its inputs by extension. Source files end with the case-insensitive extensions `.asm`, `.s`, or `.src`, and are preprocessed and assembled. Any other input is treated as a pre-assembled object file or library and is passed straight to the linker.

The following flags control how far the pipeline runs and how it behaves:

- `-p` stops after preprocessing and emits the preprocessed source.
- `-a` stops after assembling. Object files are kept, and non-source inputs are ignored. When `-o` and more than one source file are given together, the resulting objects are bundled into an archive at the output path.
- `-e <extension ...>` enables architecture extensions in the assembler, chosen from `dynamichw`, `syscall`, and `privilege`. No extensions are enabled by default.
- `-D <key>=<val>` defines a preprocessor macro before assembling.
- `-o <file>` sets the output path.
- `-l <options>` passes a string of extra options through to the linker.

### 2.1.3 Assembler Semantics

#### 2.1.3.1 Instructions

`misa-as` supports the instruction semantics defined in the instruction "Usages" of the ISA chapter. Opcodes and instruction operands are not comma-separated, and are case-insensitive.

#### 2.1.3.2 Labels

Labels are programmer-defined symbols in assembly source code that are resolved to specific addresses by the linker. Labels are C-style identifiers (containg the characters '_', a-z, A-Z, and 0-9, but not starting with 0-9) which end with a colon (':').

Label names are global to a binary at the linking stage, and can only be defined once. Assembly written for the MISA standard library and other system code reserves double-underscore prefix `__` for "local" labels. The recommended convention local labels in created by compilers and non-library assembly is a single-underscore prefix `_`. In both cases, local labels should use the name of their containing function as a discriminant. Compilers are encouraged to support name mangling for global function labels.

During linking, labels used as instruction operands will be resolved to the address of the first instruction after the label. For example, consider the following code:

```
    SET RA 0
    SET RB 10
loop:
    SET2 RYZ loop
    JMP LESS RA RB
```

In this case, the label `loop` has the relative address in this code of 0x0004, and refers to the `SET2` instruction.

#### 2.1.3.3 Assembler Directives

`misa-as` supports several assembler directives that change the behavior of the assembler and its code generation. Directives are C-style reserved identifiers that begin with a period ('.') and may take one or more operands.

The following directives are currently supported by `misa-as`:

- `.word <Word8>`: Emits the raw byte `<Word8>` in the generated code at the location of the directive
- `.array <Word8> [Word8...]`: Emits the raw bytes `<Word8> [Word8...]` in the generated code at the location of the directive. Earlier bytes in the array definition will be placed at lower addresses. `.array` is equivalent to many consecutive `.word` directives
- `.addr <Label>`: Emits the two-byte address of the label `<Label>` after linking at the location of the directive. If the linker places `<Label>` at `<HiByte> - <LoByte>`, this directive is equivalent to `.word <LoByte>  .word <HiByte>`.
- `.ascii "<String>"`: Emits the bytes of the string `<String>` using the ASCII encoding at the location of the directive. The first letter of the string is placed at a lower address. The string is not null-terminated. Escape characters are not supported.
- `.asciiz "<String>"`: Equivalent to `.ascii "<String>"` followed immediately by `.word 0x00`. That is, the string is null-terminated with a zero byte at the higher address following the last character of the string.
- `.space <Word16>`: Reserves a block of memory of length `<Word16>`. The contents of the memory is undefined at runtime. Linker implementations may either fill the memory or simply not place other data/text in the reserved area, leaving it uninitialized.
- `.section <String>`: Defines for the linker that all the following code until the next section directive should be placed in a section called `<String>`

#### 2.1.3.4 Preprocessor Directives

`misa-as` also supports preprocessor directives, but these are not resolved by the assembler directly. See the section of the MISA preprocessor `misa-pr` for more information on syntax. Preprocessor directives are resolved by the assembler before the assembly or linking phases.

### 2.1.4 MISA Linkable Format

Object files produced by `misa-as` are in the custom `MISA_LF` linkable format, which are binary files that contain:

- The assembled source code for one or more sections defined in a single assembler translation unit.
- A list of symbols (assembler labels) defined in the translation unit, along with their addresses relative to the beginning of the object file's machine code.
- A list of symbols required (not defined) by the translation unit during the linking phase, along with the address relative to the beginning of the object file's machine code where the symbol is needed.

#### 2.1.4.1 Object File Description

Binary object files and all fields within them are encoded with little endian byte order. Each object file contains the following information:

| Length | Type | Content |
|---|---|---|
| 8 | String | The string `MISA_LF`$_\sqcup$, where `M` is the least significant byte and $_\sqcup$ is the most significant byte. |
| 2 | Word16 | The number of sections present in the object file. |
| Variable | [Sec] | The content of each section, repeated as many times as specified in the length field. |
| 2 | Word16 | The number of strings present in the string table. |
| Variable | [(Word16, String)] | The ordered list of all string identifiers preceeded by their lengths, repeated as specified in the length field. |

#### 2.1.4.2 Object File Section Description

Each section contains the following information:

| Length | Type | Content |
|---|---|---|
| 2 | Word16 | The length in bytes of the section name string field. |
| Variable | String | The name of the section as defined with the `.section` assembler directive. |
| Variable | Code | The content of the assembled machine code for the specified section. |
| Variable | SymTable | The table of symbols defined by the code in the specified section. |
| Variable | RelocTable | The table of relocations required by the code in the specified section. |

#### 2.1.4.3 Object File Code Description

Each code table contains the following information:

| Length | Type | Content |
|---|---|---|
| 2 | Word16 | The length in bytes of the machine code. |
| Variable | [Word8] | The machine code. |

#### 2.1.4.4 Object File Symbol Table Description

Each symbol table contains the following information:

| Length | Type | Content |
|---|---|---|
| 2 | Word16 | The number of symbols in the symbol table. |
| Variable | [Sym] | The list of symbols. |

Each entry in the symbol table contains the following information:

| Length | Type | Content |
|---|---|---|
| 2 | Word16 | The address relative to the start of the code table that the symbol points to. |
| 2 | Word16 | The 0-aligned index into the object file's string table where the symbol's name can be found. |

#### 2.1.4.5 Object File Relocation Table Description

Each relocation table contains the following information:

| Length | Type | Content |
|---|---|---|
| 2 | Word16 | The number of relocations in the relocation table. |
| Variable | [Reloc] | The list of relocations. |

Each entry in the relocation table contains the following information:

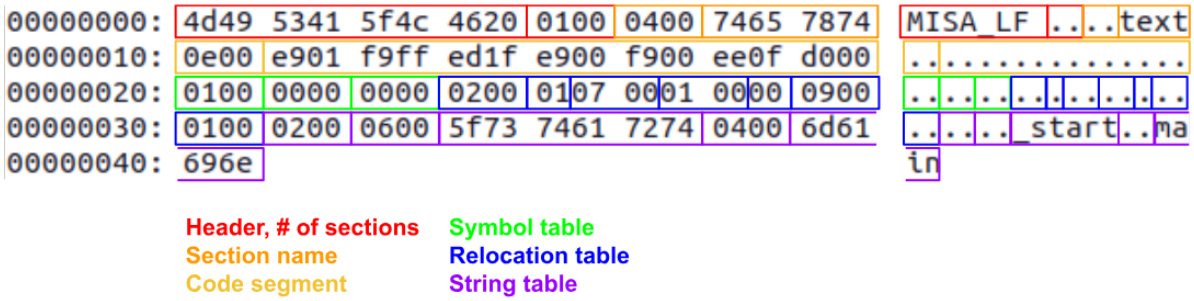
| Length | Type | Content |
|---|---|---|
| 1 | Word8 | Whether the byte at the relocation address must be filled with the high (0x01) or low (0x00) half of the relocated symbol's address. |
| 2 | Word16 | The address relative to the start of the code table of the byte that needs to be filled by the linker. |
| 2 | Word16 | The 0-aligned index into the object file's string table where the symbol's name can be found. |

#### 2.1.4.6 Object File Example

Consider the following code stub that has code, exported symbols, and required relocations:

```
    .section text
_start:                     // _start is a symbol
    SET2 RSCRATCH 0x01FF
    WSR SADDR RSCRATCH
    CALL main               // main is a relocation
    HALT RT
```

Assembling this code to an object file with `misa-as -a start.S -o start.o` will produce the following object code:



## 2.2 Linker

`misa-ld` is the reference linker for the `MISA_LF` linkable format of object files. It places the code sections of one or more object files into a flat binary, resolving symbol positions and applying relocations along the way. Archive files produced by `misa-ar` are recognized by the case-insensitive `.a` extension. The linker extracts them first and links the object files they contain.

The linker always emits a flat binary rather than a new object file. The binary spans the entire 64 KiB address space of the MISA architecture, so it can be loaded directly into the memory of a MISA processor. The output path defaults to `memory.bin` and is set with `-o`.

When `-g <file>` is given, the linker also emits an object file that holds only a symbol table with the absolute address of every placed symbol. This debug object is meant for tools that resolve symbol names to addresses, such as the simulator described in Chapter 3.

### 2.2.1 Memory Map Files

A memory map file is a small linker script that tells the linker where to place each section in the address space. Each entry gives an address range and the sections that belong in it. The grammar is:

```
memmap  ::= ("section" number "-" number ":" (section)* ";")+
number  ::= 0x(0-9a-fA-F)+
section ::= (_a-zA-Z)(_a-zA-Z0-9)*
```

A memory map file is selected with `-M <path>`. When none is given, the linker uses the following default map:

```
section 0x0000 - 0x00FF : direct      ;  // Zero pages
section 0x0100 - 0x01FF :             ;  // Stack
section 0x0200 - 0x7FFF : data bss    ;  // RAM
section 0x8000 - 0xBFFF :             ;  // Memory-mapped IO
section 0xC000 - 0xFFEF : text rodata ;  // Program ROM
section 0xFFF0 - 0xFFFF : vectors     ;  // Vector table
```

## 2.3 Other Tools

The toolchain includes several smaller tools for preprocessing source, bundling objects, and inspecting binaries.

### 2.3.1 Preprocessor

`misa-pr` is the source-agnostic preprocessor that runs before the assembler. It resolves `#` directives and emits a file with the directives removed. A file with no directives passes through unchanged. Preprocessing runs in two passes. Inclusions are resolved first, then macros are expanded over the combined content from top to bottom.

#### 2.3.1.1 Includes

The `#include` directive pastes the contents of another file in place of the directive:

```
#include "<path>"
```

Inclusions are resolved recursively, so an included file may include further files. The preprocessor looks for `<path>` first relative to the current directory, then in each directory listed in the `MISA_PR_INCLUDE_PATH` environment variable. That variable is a colon-separated list of directories, searched in order.

#### 2.3.1.2 Macros

An object-like macro replaces every later token that matches its name with a body:

```
#macro <name> <body> #endm
```

A function-like macro takes a parenthesized list of space-separated arguments and substitutes them into its body at each call:

```
#macro <name>(<arg> ...) <body> #endm
```

Such a macro is called as `<name>(<arg> ...)` with space-separated arguments, and is expanded only when the number of arguments matches the definition. Macros may be redefined, and the most recent definition above a use takes effect. A macro can also be defined for the whole file on the command line with `-D <key>=<val>`, which behaves like an object-like macro.

### 2.3.2 Archiver

`misa-ar` bundles several `MISA_LF` object files into a single archive, and extracts them again. Archives are used to distribute libraries of object files that the linker consumes directly. Only the object file contents are preserved, so other filesystem metadata is not restored on extraction.

`misa-ar` takes one of two mutually exclusive subcommands:

- `misa-ar c <objfile ...> [-o <file>]` creates an archive from the given objects. The output defaults to `library.a`.
- `misa-ar x <archive> [-o <dir>]` extracts the objects from an archive into a directory, which defaults to the current directory.

The linker and `misa-nm` accept archives directly and extract them on their own.

#### 2.3.2.1 Archive File Description

Archive files use the `MISA_AR` container format. As with the object format, all fields are encoded with little endian byte order. Each archive contains the following information:

| Length | Type | Content |
|---|---|---|
| 8 | String | The string `MISA_AR`␣, where `M` is the least significant byte and ␣ is the most significant byte. |
| 2 | Word16 | The number of objects present in the archive. |
| Variable | [NamedObject] | The named objects, repeated as many times as specified in the count field. |

Each named object pairs an object file with the name it is restored under:

| Length | Type | Content |
|---|---|---|
| 2 | Word16 | The length in bytes of the object name string. |
| Variable | String | The object file name, used as the file name on extraction. |
| Variable | Object | A full MISA_LF object file, laid out as described above. |

### 2.3.3 Name Lister

`misa-nm` lists the symbols in `MISA_LF` object files and `MISA_AR` archives. With no file arguments it prints nothing. Archives are extracted first, and the symbols of each archived object are listed in turn.

The listing begins with a `File "<path>"` line, then one `Section <name>:` header for each section in the object. Within a section, a defined symbol that the object exports is marked `<-`, and a required symbol that the object imports is marked `->`. Each line shows the symbol or relocation address within the object in hexadecimal, followed by the symbol name.

A required symbol also carries a relocation marker. `(hi)` means the relocation fills bits 15:8 of the symbol value, and `(lo)` means it fills bits 7:0. Single-byte relocations default to `(lo)`.

For the object file from the earlier example, `misa-nm start.o` prints:

```
File "start.o"

  Section text:

    <- 0000 _start
    -> 0007 main (hi)
    -> 0009 main (lo)
```

Here `_start` is defined at the start of the code, and the two `main` entries are the high and low relocation holes left by the `CALL main` expansion.

### 2.3.4 Object Dumper

`misa-objdump` disassembles flat binaries and `MISA_LF` object files back into MISA assembly. The file format is inferred from the `MISA_LF` magic header, and any other file is treated as a flat binary. The format can be forced with `-f <format>`, either `misa_lf` or `binary`.

Disassembly is requested with one of two flags. `-d` disassembles the sections expected to hold code, which applies to object files. `-D` disassembles the whole file and is required for a flat binary, since a flat binary has no section structure. The `--start-address` and `--stop-address` options restrict the output to a range.

Each disassembled line shows the address, the raw instruction bytes, and the decoded instruction. An object file carries its own symbol and relocation tables, so labels and relocated immediates appear directly in its disassembly. For a flat binary, a debug object file passed with `-g <file>` supplies those same annotations.

Disassembling the earlier object file with `misa-objdump -d start.o` prints:

```
                        .section text
                        _start:
0x0000  E9 01 F9 FF         SET2 RSCRATCH0 RSCRATCH1 0x1FF
0x0004  ED 1F               WSR SADDR RSCRATCH0 RSCRATCH1
```

```
0x0006  E9 00 F9 00         SET2 RSCRATCH0 RSCRATCH1 main
0x000A  EE 0F               JAL ALWAYS RSCRATCH0 RSCRATCH1
0x000C  D0 00               HALT RT
```

The `CALL main` pseudo instruction expands into a `SET2` and `JAL` pair, and the unresolved `main` relocation is shown by name in the `SET2` immediate.

# 3 Simulator And Hardware Reference

This chapter covers the reference simulator for the MISA architecture and the planned hardware implementation.

## 3.1 Functional Simulator

`misa-sim` is the functional reference implementation of the MISA instruction set. It executes the flat binaries produced by the linker and models the processor as a single-cycle machine. The simulator is intended to match Chapter 1 exactly, so it serves as an executable reference for the behavior of every instruction and how those instructions interact with memory, processor state, etc.

The simulator comes with an interactive GDB-style debugger for loading, inspecting, and executing programs. The debugger is documented in the next section.

### 3.1.1 Loading And Reset

A program is loaded into memory as a flat binary starting at a byte offset, which defaults to `0x0000`. Loading copies bytes from the file into memory until either the file ends or the top of memory is reached. The linker emits an image that spans the whole address space including the vector table, so a linked binary is normally loaded at offset `0x0000`.

The simulator resets after a binary is loaded. Reset reads the reset vector stored in little-endian at `0xFFFE - 0xFFFF` and initializes the program counter to that address. The processor is then ready to execute instructions.

### 3.1.2 Instruction Execution

Each step fetches the 16-bit little-endian instruction at the program counter, and advances the program counter by two bytes before executing. Execution of each instruction follows the semantics in Chapter 1. When the program halts, the simulator returns to its reset state and stops.

A faulting instruction discards its partial result, records the reason and type in the `CAUSE` register, and sets the program counter to the fault handler address stored at `0xFFFC - 0xFFFD`.

Execution then continues in the fault handler. This matches the fault behavior defined in Chapter 1.

### 3.1.3 Address Space

The reference simulator implements the entire 16-bit address space. All 65536 bytes are mapped, and memory accesses wrap at `0xFFFF`, so an access past the top of memory returns to the bottom. Because no region is unmapped, the simulator never raises the "Address overflow" memory usage fault from Chapter 1. That fault is reserved for implementations with a narrower bus or a memory protection unit. The simulator does still raise the "PC overflow" fault when the program counter would leave the 16-bit range during an increment.

## 3.2 Interactive Debugger

The simulator is driven by an interactive debugger, started with `misa-sim [binary]`. The optional binary is loaded at startup as though passed to the `load` command. The debugger presents a `(misa)` prompt and reads one command per line. Command history is saved between sessions.

Type `help` to list the available commands, or `help <command>` for the detailed help of a single command. Most commands accept short aliases, listed below. Arguments in angle brackets are required, and arguments in square brackets are optional.

| Command | Aliases | Usage | Description |
|---|---|---|---|
| `load` | | `load <file> [offset]` | Load a binary image into memory at `[offset]` (default `0x0000`) and reset the simulator. |
| `symbol-file` | `symbol`, `sym` | `symbol-file <file>` | Import a symbol table from an object file for use by later commands. |
| `disassemble` | `disass` | `disassemble [start,[stop\|+count]]` | Disassemble memory, defaulting to a window from 8 bytes before to 16 bytes after the program counter. |
| `reset` | | `reset` | Reset the processor and jump to the reset vector. |
| `step` | `si`, `s` | `step [n]` | Execute at most `[n]` instructions (default `1`), stopping early on a halt or breakpoint. |
| `continue` | `cont`, `c` | `continue` | Run until a breakpoint or a halt. |
| `breakpoint` | `break`, `br`, `b` | `breakpoint <addr\|sym>` | Set a breakpoint at an address or an imported symbol. |

| Command | Aliases | Usage | Description |
|---|---|---|---|
| `delete` | `del`, `d` | `delete [addr\|sym]` | Remove one breakpoint, or all breakpoints when given no argument. |
| `info` | `i` | `info <topic>` | Show simulator state. The topics are `registers` and `breakpoints`. |
| `examine` | `exam`, `x` | `examine <addr> [count]` | Dump `[count]` bytes of memory (default `16`) starting at `<addr>`. |
| `quit` | `q` | `quit` | Save history and exit the simulator. |

Breakpoints must be placed at the lower byte of an instruction to be hit. A symbol argument to `breakpoint`, `delete`, or `disassemble` is resolved to an address immediately using the most recently imported symbol file, so later imports do not move an existing breakpoint. Disassembly and symbol import both require the referenced files to still exist on disk.

## 3.3 Hardware Reference

The ultimate goal of the MISA project is to design, verify, and manufacture an ASIC that runs a complete implementation of the architecture. Tapeout shuttle programs that offer such a service often carry restrictive area and pinout requirements, so the intended design is a single-cycle processor sized for those constraints.

The hardware reference is planned work.